

Task on s3, CDN, Route 53

1. Configure VPC peering in cross regions.

Objective: Establish private network connectivity between two VPCs located in different AWS Regions.

Implementation Steps

1. Open the AWS Management Console.
2. Select the first Region, for example N. Virginia (us-east-1).
3. Go to VPC → Your VPCs and create/select VPC-1.
4. Note the VPC CIDR, for example:
 - VPC-1: 172.31.0.0/24
5. Switch to the second Region, for example ohio (us-east-2).
6. Create/select VPC-2.
7. Ensure the VPC CIDRs do not overlap, for example:
 - VPC-2: 192.168.0.0/24
8. In VPC-1 Region, go to VPC → Peering connections.
9. Click Create peering connection.
10. Provide:
 - Name: Cross-Region-VPC-Peering
 - Requester VPC: VPC-1
 - Acceptor Region: us-east-1
 - Acceptor VPC: VPC-2
11. Create the peering connection.
12. Switch to the acceptor Region.
13. Go to VPC → Peering connections.
14. Select the pending peering connection.
15. Choose Actions → Accept request.
16. Update the route table of VPC-1:
 - Destination: 172.31.0.0/24
 - Target: VPC Peering Connection

17. Update the route table of VPC-2:

- Destination: 192.168.0.0/24
- Target: VPC Peering Connection

18. Update the security groups to allow the required traffic.

19. Launch test EC2 instances in both VPCs.

20. Test private connectivity using the private IP addresses.

The image displays two screenshots of the AWS Management Console, specifically the VPC dashboard, showing details for two different VPCs.

Top Screenshot: vpc-0926f9e7dd991735e / Default-vpc

- VPC ID:** vpc-0926f9e7dd991735e
- State:** Available
- Block Public Access:** Off
- DNS hostnames:** Enabled
- DNS resolution:** Enabled
- Tenancy:** default
- DHCP option set:** dopt-0b01bcb1a413a639
- Main network ACL:** acl-09568d09855860376
- Default VPC:** Yes
- IPv4 CIDR:** 172.31.0.0/16
- Main route table:** rtb-07966b87459680edc / default-RT
- IPv6 CIDR:** -
- Route 53 Resolver DNS Firewall rule groups:** -
- IPv6 pool:** -
- Owner ID:** 281712468655
- Encryption control ID:** -
- Network Address Usage metrics:** Disabled
- Encryption control mode:** -

Bottom Screenshot: vpc-083bdd85d37857186 / my-vpc-01

- VPC ID:** vpc-083bdd85d37857186
- State:** Available
- Block Public Access:** Off
- DNS hostnames:** Disabled
- DNS resolution:** Enabled
- Tenancy:** default
- DHCP option set:** dopt-0c994995a0e6f7902
- Main network ACL:** acl-056f308f5f837b9e2
- Default VPC:** No
- IPv4 CIDR:** 192.168.0.0/24
- Main route table:** rtb-00a070a723624ca66
- IPv6 CIDR (Network border group):** -
- Route 53 Resolver DNS Firewall rule groups:** -
- IPv6 pool:** -
- Owner ID:** 281712468655
- Encryption control ID:** -
- Network Address Usage metrics:** Disabled
- Encryption control mode:** -

us-east-1.console.aws.amazon.com/vpcconsole/home?region=us-east-1#PeeringConnectionDetails:VpcPeeringConnectionId=pcx-09beaeb8284e1a5fb

pcx-09beaeb8284e1a5fb / peering-virgina-ohio

Details Info

Requester owner ID
281712468655

Peering connection ID
pcx-09beaeb8284e1a5fb

Status
Active

Expiration time
-

Accepter owner ID
281712468655

Requester VPC
vpc-083bdd85d37857186 / my-vpc-01

Requester CIDRs
192.168.0.0/24

Requester Region
N. Virginia (us-east-1)

VPC Peering connection ARN
arn:aws:ec2:us-east-1:281712468655:vpc-peering-connection/pcx-09beaeb8284e1a5fb

Accepter VPC
vpc-0926f9e7dd991735e

Accepter CIDRs
172.31.0.0/16

Accepter Region
Ohio (us-east-2)

DNS Route tables Tags

CloudShell Agent Toolkit for AWS Feedback Console Mobile App © 2026, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

us-east-2.console.aws.amazon.com/vpcconsole/home?region=us-east-2#PeeringConnectionDetails:VpcPeeringConnectionId=pcx-09beaeb8284e1a5fb

pcx-09beaeb8284e1a5fb

Details Info

Requester owner ID
281712468655

Peering connection ID
pcx-09beaeb8284e1a5fb

Status
Active

Expiration time
-

Accepter owner ID
281712468655

Requester VPC
vpc-083bdd85d37857186

Requester CIDRs
192.168.0.0/24

Requester Region
N. Virginia (us-east-1)

VPC Peering connection ARN
arn:aws:ec2:us-east-2:281712468655:vpc-peering-connection/pcx-09beaeb8284e1a5fb

Accepter VPC
vpc-0926f9e7dd991735e / Default-vpc

Accepter CIDRs
172.31.0.0/16

Accepter Region
Ohio (us-east-2)

DNS Route tables Tags

CloudShell Agent Toolkit for AWS Feedback Console Mobile App © 2026, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

us-east-1.console.aws.amazon.com/vpcconsole/home?region=us-east-1#RouteTableDetails:RouteTableId=rtb-00a070a723624ca66

rtb-00a070a723624ca66

Updated routes for rtb-00a070a723624ca66 successfully

VPC
vpc-083bdd85d37857186 | my-vpc-01

Owner ID
281712468655

Routes Subnet associations Edge associations Route propagation Tags Related resources

Routes (3)

Filter routes

Destination	Target	Status	Propagated	Route Origin
0.0.0.0/0	lgw-0406dd37677767...	Active	No	Create Route
172.31.0.0/16	pcx-09beaeb8284e1a...	Active	No	Create Route
192.168.0.0/24	local	Active	No	Create Route Table

CloudShell Agent Toolkit for AWS Feedback Console Mobile App © 2026, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

us-east-2.console.aws.amazon.com/vpcconsole/home?region=us-east-2#RouteTableDetails:RouteTableId=rtb-07966b87459680edc

VPC > Route tables > rtb-07966b87459680edc

VPC dashboard

Filter by VPC

AWS Global View

Virtual private cloud

Your VPCs

Subnets

Route tables

Internet gateways

Egress-only internet gateways

DHCP option sets

Elastic IPs

Managed prefix lists

NAT gateways

Details

Route table ID: rtb-07966b87459680edc

VPC: vpc-0926f9e7dd991735e | Default-vpc

Main: Yes

Owner ID: 281712468655

Explicit subnet associations: subnet-0a8d1ce0712a232ff

Edge associations: -

Routes (2)

Destination	Target	Status	Propagated	Route Origin
172.31.0.0/16	local	Active	No	Create Route Table
192.168.0.0/24	pcx-09beaeb8284e1a...	Active	No	Create Route

us-east-1.console.aws.amazon.com/ec2/home?region=us-east-1#InstanceDetails:instanceId=i-02b6fb4eabb204524

EC2 > Instances > i-02b6fb4eabb204524

Instance summary for i-02b6fb4eabb204524 (ec2-sample)

Updated 2 minutes ago

Instance ID: i-02b6fb4eabb204524

Public IPv4 address: 100.58.91.54 | open address

Private IPv4 addresses: 192.168.0.9

Instance state: Running

Public DNS: -

Private IP DNS name (IPv4 only): ip-192-168-0-9.ec2.internal

Instance type: t3.micro

Auto-assigned IP address: 100.58.91.54 [Public IP]

VPC ID: vpc-083bdd85d37857186 (my-vpc-01)

IAM role: -

Subnet ID: -

Auto Scaling Group name: -

us-east-2.console.aws.amazon.com/ec2/home?region=us-east-2#InstanceDetails:instanceId=i-0333cf01352256339

EC2 > Instances > i-0333cf01352256339

Instance summary for i-0333cf01352256339 (ec2-sample-server)

Updated less than a minute ago

Instance ID: i-0333cf01352256339

Public IPv4 address: -

Private IPv4 addresses: 172.31.4.195

Instance state: Running

Public DNS: -

Private IP DNS name (IPv4 only): ip-172-31-4-195.us-east-2.compute.internal

Instance type: t3.micro

Auto-assigned IP address: -

VPC ID: vpc-0926f9e7dd991735e (Default-vpc)

Elastic IP addresses: -

AWS Compute Optimizer finding: Opt-in to AWS Compute Optimizer for recommendations. | Learn more

```
91738@IDEAPAD MINGW64 ~/Downloads
$ ssh -i linux-testing.pem ec2-user@100.58.91.54
The authenticity of host '100.58.91.54 (100.58.91.54)' can't be established.
ED25519 key fingerprint is SHA256:5xFqs10baldxMPbjEg/xynHvy4ZbQVcsaG8CsHovra0.
This key is not known by any other names
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '100.58.91.54' (ED25519) to the list of known hosts.

#
~\##### Amazon Linux 2023
~~\#####
~~\#####
~~\###|
~~\##/
~~V~' -> https://aws.amazon.com/linux/amazon-linux-2023
~~~
~~~'-'
~~~\_/
~~~/_m/'

[ec2-user@ip-192-168-0-9 ~]$ ping 172.31.4.195
PING 172.31.4.195 (172.31.4.195) 56(84) bytes of data.
64 bytes from 172.31.4.195: icmp_seq=1 ttl=127 time=11.5 ms
64 bytes from 172.31.4.195: icmp_seq=2 ttl=127 time=11.5 ms
64 bytes from 172.31.4.195: icmp_seq=3 ttl=127 time=11.5 ms
64 bytes from 172.31.4.195: icmp_seq=4 ttl=127 time=11.5 ms
64 bytes from 172.31.4.195: icmp_seq=5 ttl=127 time=11.6 ms
64 bytes from 172.31.4.195: icmp_seq=6 ttl=127 time=11.5 ms
^C
--- 172.31.4.195 ping statistics ---
6 packets transmitted, 6 received, 0% packet loss, time 5008ms
rtt min/avg/max/mdev = 11.465/11.521/11.615/0.050 ms
[ec2-user@ip-192-168-0-9 ~]$
```

2. Purchase one domain from GoDaddy.

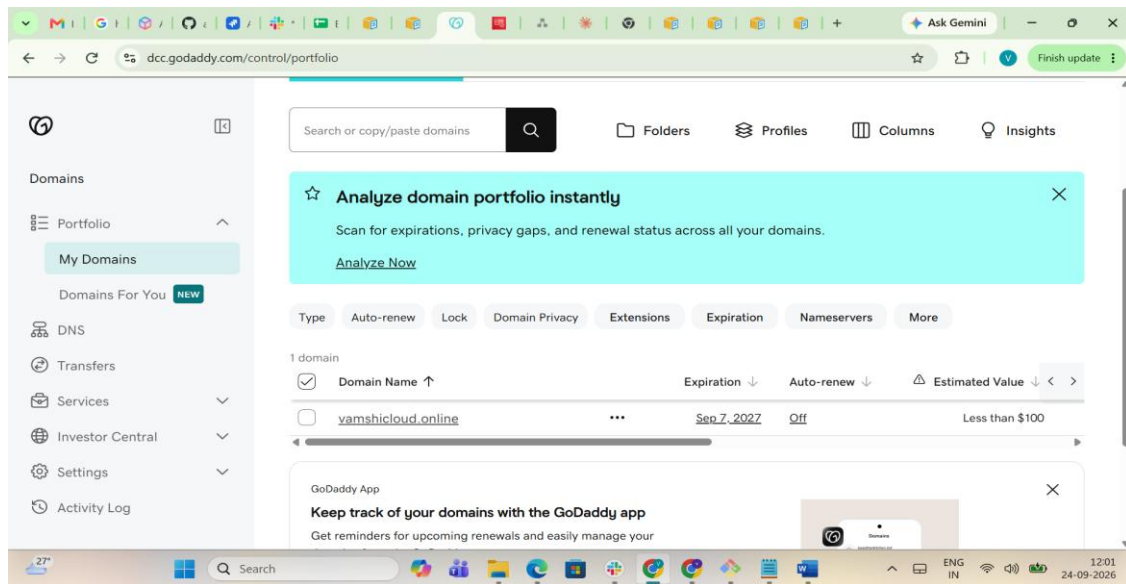
Objective: Purchase a domain that will be used to access the website through a custom domain name.

Implementation Steps

1. Open [GoDaddy](#).
2. Search for the required domain name.
3. Select an available domain.
4. Add the domain to the cart.
5. Complete the registration and payment process.
6. Open My Products → Domains.
7. Select the purchased domain.
8. Open DNS Management.
9. Keep the DNS management page available because the AWS Route 53 name servers will be configured later.

Example:

Domain: vamshicloud.online



3. Deploy static website in S3.

Objective: Host a static website using an Amazon S3 bucket.

Implementation Steps

1. Open the AWS Management Console.
2. Go to S3.
3. Click Create bucket.
4. Enter a globally unique bucket name - yamshicloud-static-2026.
5. Select the required AWS Region.
6. Create the bucket.
7. Open the bucket.
8. Upload the website files:
 - index.html
9. Open Properties.
10. Locate Static website hosting.
11. Enable static website hosting.
12. Select:
 - Hosting type: Host a static website
 - Index document: index.html

13. Save the configuration.

14. Test the S3 website endpoint.

The screenshot displays the AWS Management Console interface for the bucket 'vamshicloud-static-2026'. The 'Objects' tab is active, showing a list of two objects: 'error.html' and 'index.html'. The 'Static website hosting' section is also visible, showing that it is enabled and providing the bucket website endpoint: <http://vamshicloud-static-2026.s3-website-us-east-1.amazonaws.com>.

Name	Type	Last modified	Size	Storage class
error.html	html	September 24, 2026, 12:48:28 (UTC+05:30)	23.0 B	Standard
index.html	html	September 24, 2026, 12:54:32 (UTC+05:30)	164.0 B	Standard

Static website hosting
Use this bucket to host a website or redirect requests. [Learn more](#)

S3 static website hosting
Enabled

Hosting type
Bucket hosting

Bucket website endpoint
When you configure your bucket as a static website, the website is available at the AWS Region-specific website endpoint of the bucket. [Learn more](#)
<http://vamshicloud-static-2026.s3-website-us-east-1.amazonaws.com>

Hello from S3!

My static website is working.

4. Create a CDN and attach one SSL certificate.

Objective: Distribute the S3 website globally using a CDN and secure it using HTTPS.

For AWS, use Amazon CloudFront as the CDN.

Implementation Steps

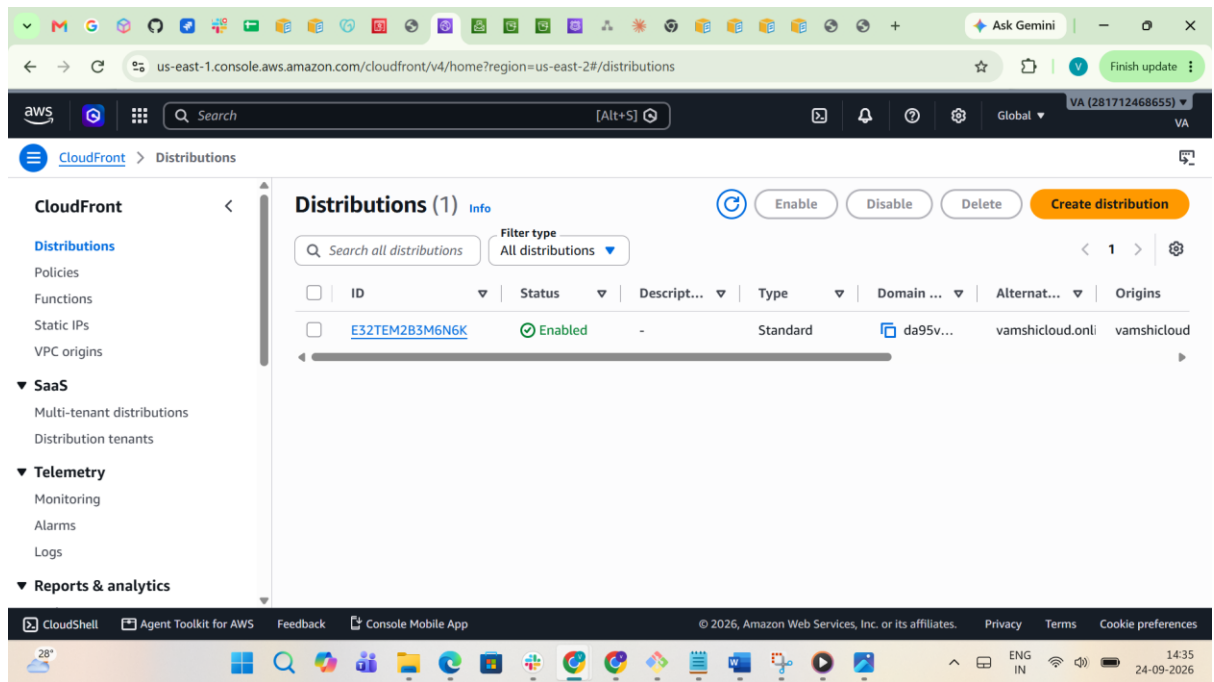
1. Open AWS Certificate Manager (ACM).
2. Select the Region required for the certificate.
3. Click Request certificate.
4. Select Public certificate.
5. Enter the domain name.

For example:

vamshicloud.online

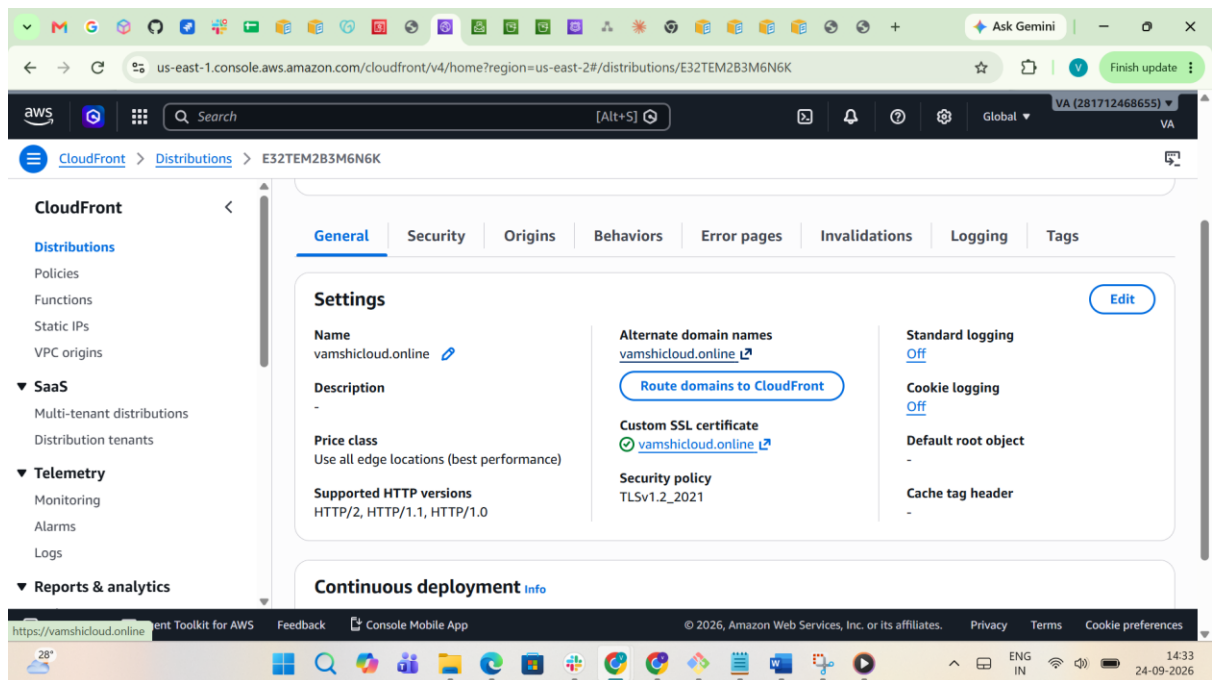
www.vamshicloud.online

6. Select DNS validation.
7. Create/add the DNS validation records.
8. Wait until the certificate status becomes Issued.
9. Open CloudFront.
10. Click Create distribution.
11. Configure the S3 bucket as the origin.
12. Configure the default root object:
index.html
13. Under Settings, add the custom domain under Alternate domain names (CNAMEs).
14. Select the ACM SSL certificate.
15. Configure the viewer protocol policy:
Redirect HTTP to HTTPS
16. Create the CloudFront distribution.
17. Wait until the distribution status becomes Deployed.
18. Copy the CloudFront distribution domain name.



Hello from S3!

My static website is working.



5. Create a Route 53 hosted zone and map the domain with the CDN.

Objective: Connect the GoDaddy domain with the CloudFront distribution using Amazon Route 53.

Implementation Steps

1. Open Amazon Route 53.
2. Go to Hosted zones.
3. Click Create hosted zone.
4. Enter the domain name.

Example:

vamshicloud.online

5. Select:

Public hosted zone

6. Create the hosted zone.
7. Route 53 will provide four Name Server (NS) records.
8. Copy the four Route 53 name servers.
9. Open GoDaddy → Domain → DNS/Nameservers.
10. Change the GoDaddy nameservers to the Route 53 nameservers.
11. Save the changes.
12. Return to Route 53.
13. Open the hosted zone.
14. Click Create record.
15. Create an A record.
16. Configure:
 - Record name: blank for root domain
 - Record type: A
 - Alias: Yes
 - Route traffic to: CloudFront distribution
17. Select the CloudFront distribution.

18. Create the record.

19. If required, create another record for:

www.vamshicloud.online

20. Wait for DNS propagation.

Expected Result:

vamshicloud.online



Route 53

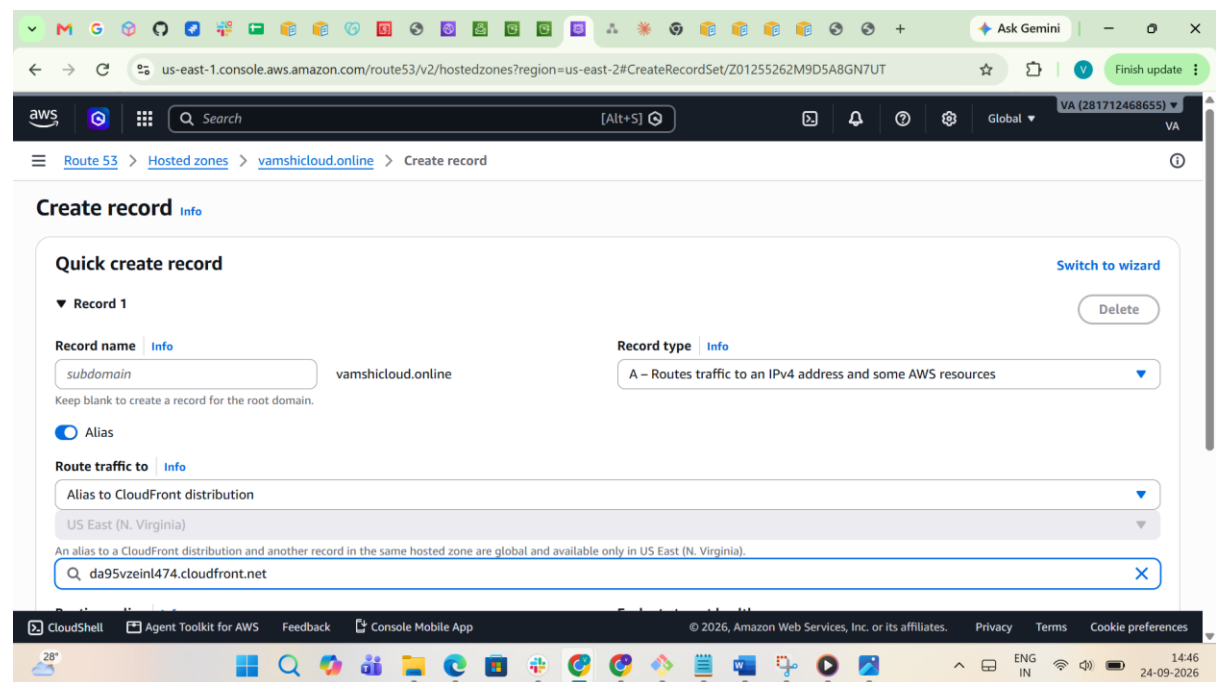


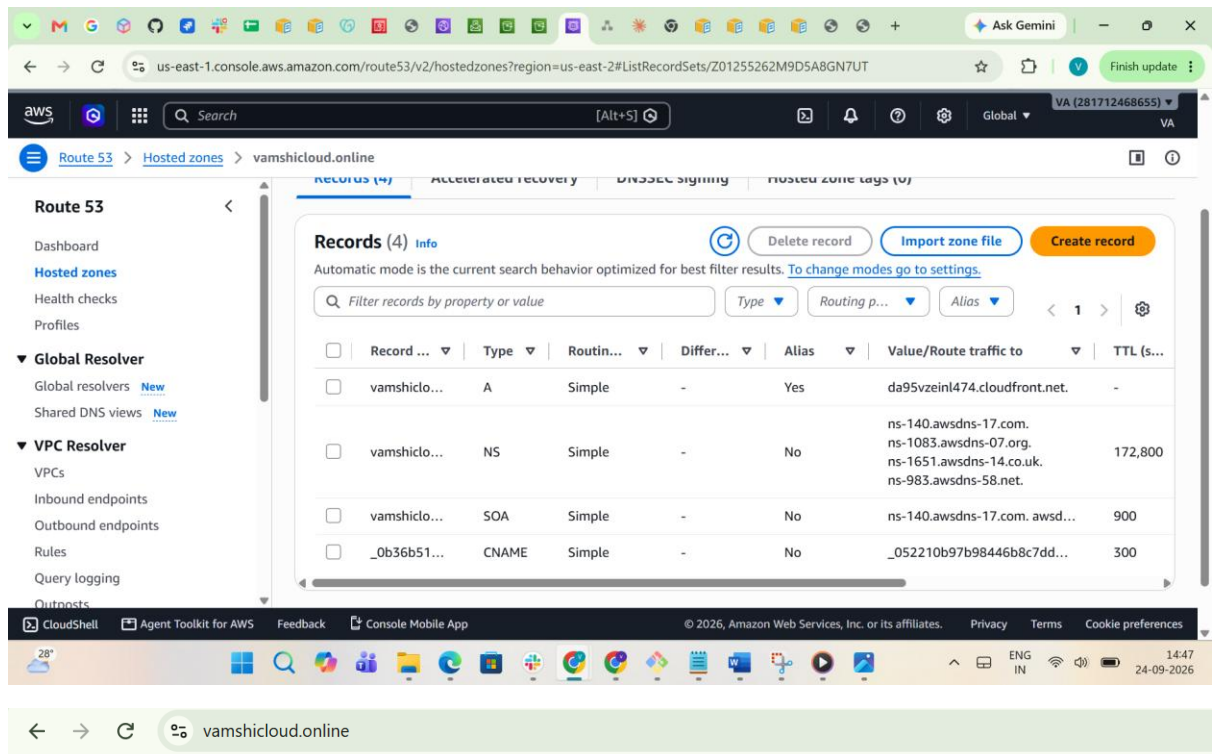
CloudFront



S3

The custom domain should open the website through CloudFront.





Hello from S3!

My static website is working.

6. Update the index.html in the S3 bucket and ensure the updated file is accessible using the domain name.

Objective: Verify that changes made to the S3 website are reflected through the CloudFront custom domain.

Implementation Steps

1. Open the S3 bucket.
2. Locate index.html.
3. Modify the website content.

For example:

```
<h1>Welcome to My AWS Website</h1>
```

```
<p>Website successfully deployed using S3 and CloudFront.</p>
```

4. Save the updated file.

5. Upload the new index.html to the S3 bucket.
6. Verify that the file was successfully uploaded.
7. Open the CloudFront distribution.
8. If CloudFront is caching the old version, go to Invalidations.
9. Create a new invalidation.
10. Enter:

/*

11. Submit the invalidation.
12. Wait until the invalidation status becomes Completed.
13. Open the custom domain in the browser.

Example:

<https://vamshicloud.online>

14. Verify that the updated website content is displayed.
15. Test the website using an incognito/private browser window if the old content is still displayed.

```
<!DOCTYPE html>
<html>
<head>
  <title>My AWS Website</title>
</head>
<body>
  <h1>Hello from S3!</h1>
  <p>My static website is updated and working.</p>
</body>
</html>
~
~
~
~
```

```
91738@IDEAPAD MINGW64 ~/Desktop
$ vi index.html

91738@IDEAPAD MINGW64 ~/Desktop
$ cat index.html
<!DOCTYPE html>
<html>
<head>
  <title>My AWS Website</title>
</head>
<body>
  <h1>Hello from S3!</h1>
  <p>My static website is updated and working.</p>
</body>
</html>
91738@IDEAPAD MINGW64 ~/Desktop
```

us-east-1.console.aws.amazon.com/s3/upload/vamshicloud-static-2026?region=us-east-1

Amazon S3

Upload succeeded
For more information, see the Files and folders table.

Destination	Succeeded	Failed
s3://vamshicloud-static-2026	1 file, 176.0 B (100.00%)	0 files, 0 B (0%)

Files and folders (1 total, 176.0 B)

Name	Folder	Type	Size	Status	Error
index.html	-	text/html	176.0 B	Succeeded	-

CloudShell Agent Toolkit for AWS Feedback Console Mobile App

© 2026, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

28°

← → ↻ ⚠ Not secure vamshicloud-static-2026.s3-website-us-east-1.amazonaws.com

Hello from S3!

My static website is updated and working.

us-east-1.console.aws.amazon.com/cloudfront/v4/home?region=us-east-2#/distributions/E32TEM2B3M6N6K/invalidations/details/IBIIR3...

CloudFront > Distributions > E32TEM2B3M6N6K > IBIIR3GHIU12WD3WG3R2TTQE23

CloudFront

- Distributions
- Policies
- Functions
- Static IPs
- VPC origins
- SaaS
 - Multi-tenant distributions
 - Distribution tenants
- Telemetry
 - Monitoring
 - Alarms
 - Logs
- Reports & analytics

Successfully created invalidation IBIIR3GHIU12WD3WG3R2TTQE23.

Invalidation details

Date created
September 24, 2026 at 9:29:08 AM UTC

Object paths
/*

Status
Completed

Copy to new

CloudShell Agent Toolkit for AWS Feedback Console Mobile App

© 2026, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

28°

15:02
24-09-2026



Hello from S3!

My static website is updated and working.



Hello from S3!

My static website is updated and working.

7. Share the domain name in Slack to test the connectivity.

Domain: <https://vamshicloud.online/>